

Shadow API Discovery & Vulnerability Scanner

A rule-based security scanner for detecting undocumented API endpoints
and confirming Broken Object Level Authorization vulnerabilities

HCIC-SI2026 — Problem 15 Submission

Author / Lead: Ekansh Jaiswal

Date: July 2026

Status: Working build, actively improved

Repository: github.com/ekansh-jaiswal/Shadow-API-Scanner

Live Demo: Runs locally as a command-line tool (no hosted demo)

This project does not use machine learning. Every finding comes from a plain, checkable rule, and where possible, the tool confirms the finding with a real network request instead of just guessing.

1 Executive Overview

Objective: Build a security scanner, with no machine learning involved, that finds "Shadow APIs" (parts of a system that quietly receive traffic but were never documented), and then checks whether they're actually exploitable.

The Challenge: Digital health systems, like India's ABDM-style setups, expose a lot of web APIs. Over time, some of these get forgotten: an old debug route nobody removed, a private link used by some internal dashboard, a "temporary" endpoint that ended up in production and stayed there. These are dangerous exactly because nobody remembers they exist. If nobody knows about it, nobody is checking it, even if it's still handling sensitive health data.

The Solution: A Python command-line tool that: (1) reads web server logs to see every endpoint that's actually getting hit, (2) compares that against an OpenAPI spec to find anything not officially listed, (3) runs seven checks based on the OWASP API Security Top 10 (2023) against each one it finds, and (4) for endpoints that hold a specific user's data, sends a real request trying to access another user's data with it. That last part is the main point: the tool doesn't guess that something might be broken. It tries to break it, for real, and shows you exactly what happened.

Business Impact: Tested against a fake health gateway ("SwasthyaConnect") built with five endpoints deliberately left undocumented, the scanner found all five, with no wrong guesses, and proved three serious Broken Object Level Authorization (BOLA) bugs live. Each one is backed by a real captured request and response, showing one patient's Aadhaar number, diagnosis, and prescription being returned using a different patient's login. Just as important, the tool correctly left two normal, safe endpoints alone (a public health check and a public doctor listing) that an earlier version of the code had wrongly flagged. A scanner nobody trusts because it keeps crying wolf isn't much better than no scanner at all, so cutting down wrong guesses mattered as much as catching the real problems.

How to Try It: This is a command-line tool, not a hosted website, so there's no live link to click. Anyone can run it themselves by cloning the repository and following the setup steps in the README — it takes a few minutes and includes a ready-to-use fake environment for testing, so no separate account or server needs to be set up first.

2 Technical Architecture & Stack

Backend / Scripting: Python 3.11, Flask (used to run a fake API server for testing), Jinja2 (used to build the report file)

Data & Parsing: PyYAML to read the OpenAPI spec, openapi-spec-validator to check that spec is actually valid before trusting it, and a regex-based reader for Nginx-style access logs

Report Output: A single HTML file with everything built in. No external scripts, no internet connection needed to open it and click around — it works the same on any machine, offline.

System Flow:

```
[Access Logs + Auth-Header Log] + [OpenAPI Spec]
→ [Log Parser / Spec Loader]
→ [Diff Engine: Shadow / Dormant / Documented]
→ [Risk Engine: 7 OWASP checks + live BOLA probe]
→ [Scorer: weighted 0--100 risk score per endpoint]
→ [HTML Report Generator]
→ [CLI: exit codes ready for CI/CD]
```

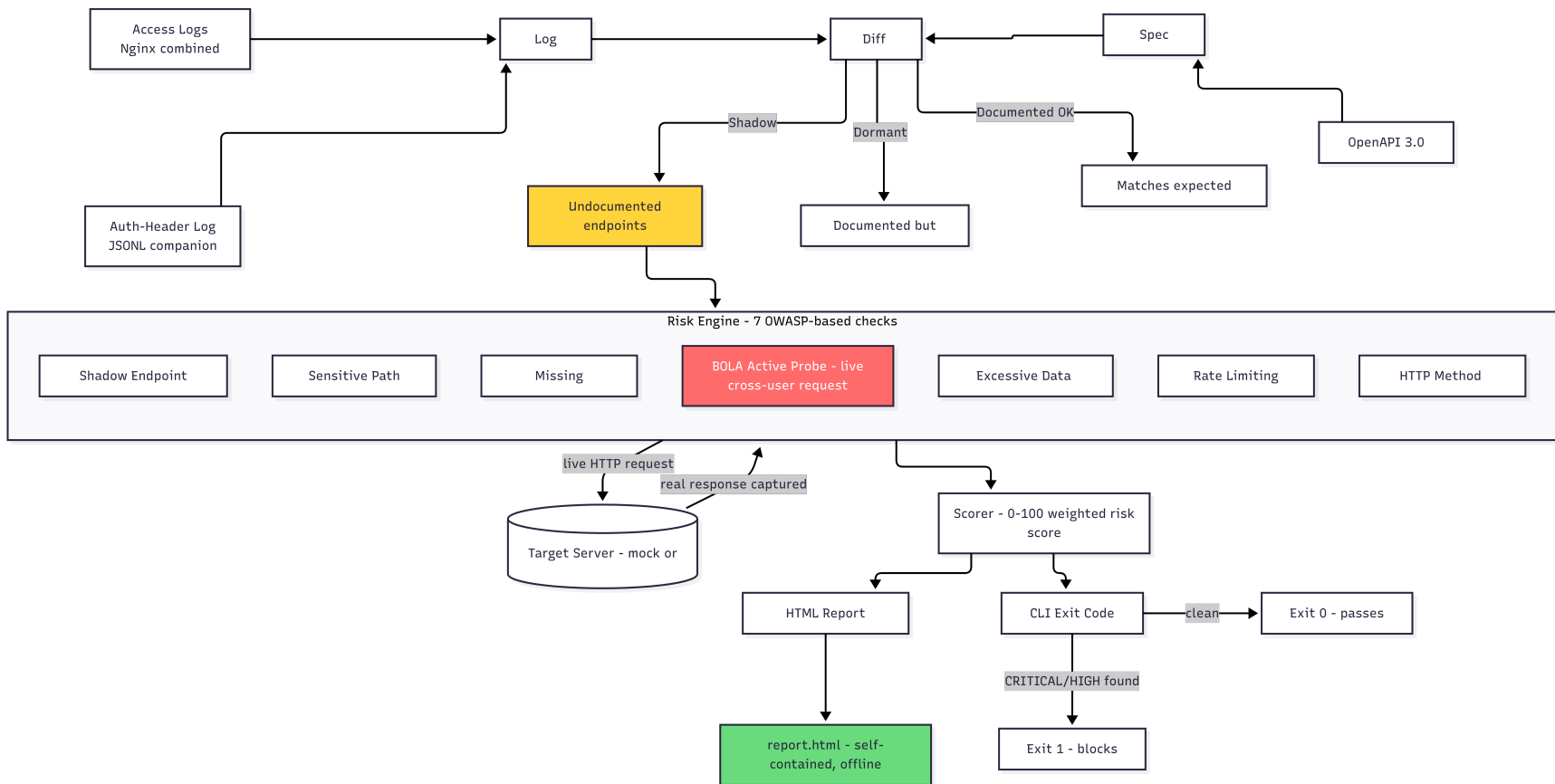


Figure 1: Shadow API Scanner — how data flows through the system

3 Key Deliverables & Features

Shadow API Discovery: Compares real traffic against an OpenAPI spec to find endpoints that were never documented. URLs like `/patient/104` and `/patient/117` are correctly matched as the same endpoint, `/patient/{id}`, instead of being counted as two separate ones just because the numbers differ.

Live BOLA Check: The scanner doesn't just guess that an endpoint might be broken based on traffic patterns. It sends a real request using one user's login token to try to reach another user's data, and records exactly what comes back. A finding is only marked CRITICAL if this actually works — the report shows the real evidence, not a prediction.

What "Rule-Based" Actually Means: Each check is a plain, human-written condition, not a trained model guessing at patterns. For example, the missing-authentication check is simply: "does the OpenAPI spec say this endpoint needs a login, and did at least one real request in the logs reach it without one?" If yes, flag it. There's no probability score, no training data, no black box — every single finding can be traced back to one exact rule, and that rule can be read directly in the code. This matters for a security tool specifically because a security team needs to know exactly why something was flagged, not just that a model was "pretty confident."

Avoiding False Alarms: An earlier version of this check fired on any endpoint with a number in the URL, which meant it wrongly flagged a public doctor listing as broken — looking up any doctor by ID is completely normal, there's no "owner" of a doctor's public profile to protect. This was caught while testing by hand and fixed by only running the cross-user check on endpoints that actually belong to a specific person's data. That removed this whole category of wrong result.

DPDP Act 2023 Notes: Findings involving personal or health data get a short note pointing to relevant sections of India's DPDP Act — Section 6 on limiting what data is collected and why, Section 8(1) on keeping data reasonably secure, and so on. These notes are written as "relevant to" or "may be worth reviewing," never "this breaks the law." The tool points out technical facts. It isn't a lawyer and doesn't try to make legal calls.

4 Implementation of Highlights & Performance

Security & Data Handling: All the demo data — patient records, Aadhaar-shaped numbers, diagnoses — is made up. No real personal data and no real systems were touched at any point while building or testing this. The live-probing feature is meant only for systems the user owns or has clear permission to test.

Handling Failure Properly: While testing by hand, a real bug turned up: running the scanner with the test server turned off still produced the exact same "confirmed" CRITICAL results as when the server was on. That's a serious problem for a tool whose whole point is "we checked this for real, we didn't just guess." The cause was an error being silently ignored inside the probe code. The fix adds a quick check at startup: if the target server can't be reached, the scanner shows a clear warning, both on screen and in the report, falls back to results based on logs alone, and the risk score drops to match — from 100 (CRITICAL) down to 61 (HIGH) in testing — instead of quietly repeating old results.

Testing: Every part of the pipeline was checked against real output, not assumed to work. This included trying to break it on purpose — missing files, a broken spec file, turning the server off mid-scan — not just testing the case where everything goes right. The exit-code behavior, which is what a real deployment pipeline would use to stop a bad release, was checked across six different scenarios. This testing caught a real bug where one setting (`--fail-on medium`) failed to catch a

CRITICAL result, because the code was checking for an exact match instead of checking "is this at least as serious as medium."

5 Challenges & Strategic Roadmap

Main Problem: Telling a real authorization bug apart from a normal public lookup, without using machine learning to do it. A simple rule like "test any endpoint with a number in the URL" ends up flagging things that were never meant to belong to any one person, like a public doctor listing.

How It Was Fixed: Added a step that checks whether an endpoint's name suggests it holds a specific person's data — using words like patient, appointment, insurance, otp, debug, prescription — so the cross-user check only runs where it actually makes sense to run it. Confirmed this worked by re-running the scan and checking that the doctor listing and health-check endpoints came back clean, while the three real bugs still got caught.

What's Next:

- A graph view showing how discovered endpoints relate to each other — shared login patterns, grouped by resource type
- Comparing two sets of logs taken at different times, to catch newly appeared shadow endpoints early
- Moving past keyword matching for the ownership check, toward actually comparing responses across different user tokens, so it still works on real APIs that don't use the same naming style as this project's test setup

6 External Validation via OWASP crAPI

Why: Test the scanner against a well-known, independently built vulnerable application — OWASP's Completely Ridiculous API (crAPI) — instead of only the custom test environment built specifically for this project. A tool that only finds bugs in its own test fixture doesn't prove much on its own. Finding bugs in something it wasn't built around is a stronger, more convincing test.

Where This Stands:

- The crAPI application has been set up and is running locally.
- It runs as several connected services (web app, backend, database) using Docker.
- All services are up, and the main web app is reachable and working.

What's Left:

- Use crAPI like a normal user would, so real traffic gets generated and logged.
- Run the scanner against that traffic and confirm it catches the authorization bugs crAPI was built to contain.

Sources

1. OWASP API Security Top 10 (2023) — <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
2. OpenAPI Specification v3.0.3 — <https://spec.openapis.org/oas/v3.0.3>
3. Nginx HTTP Log Module documentation — https://nginx.org/en/docs/http/nginx_http_log_module.html
4. Flask documentation — <https://flask.palletsprojects.com/>
5. Digital Personal Data Protection Act, 2023, Government of India — official gazette text
6. OWASP crAPI (Completely Ridiculous API) — <https://github.com/OWASP/crAPI>
7. openapi-spec-validator (PyPI) — <https://pypi.org/project/openapi-spec-validator/>